FEEDFORWARD NEURAL NETWORK FOR LANGUAGE MODELING

This code explores the use of Feedforward Neural Networks (FNNs) in language modeling. The primary objective is to build a neural network that learns word relationships and generates meaningful text sequences. The implementation is done using PyTorch, covering key aspects of Natural Language Processing (NLP), such as:

- Tokenization & Indexing: Converting text into numerical representations.
- Embedding Layers: Mapping words to dense vector representations for efficient learning.
- Context-Target Pair Generation (N-grams): Structuring training data for sequence prediction.
- Multi-Class Neural Network: Designing a model to predict the next word in a sequence.

The training process includes optimizing the model with loss functions and backpropagation techniques to improve accuracy and coherence in text generation. End result is a working FNN-based language model capable of generating text sequences.

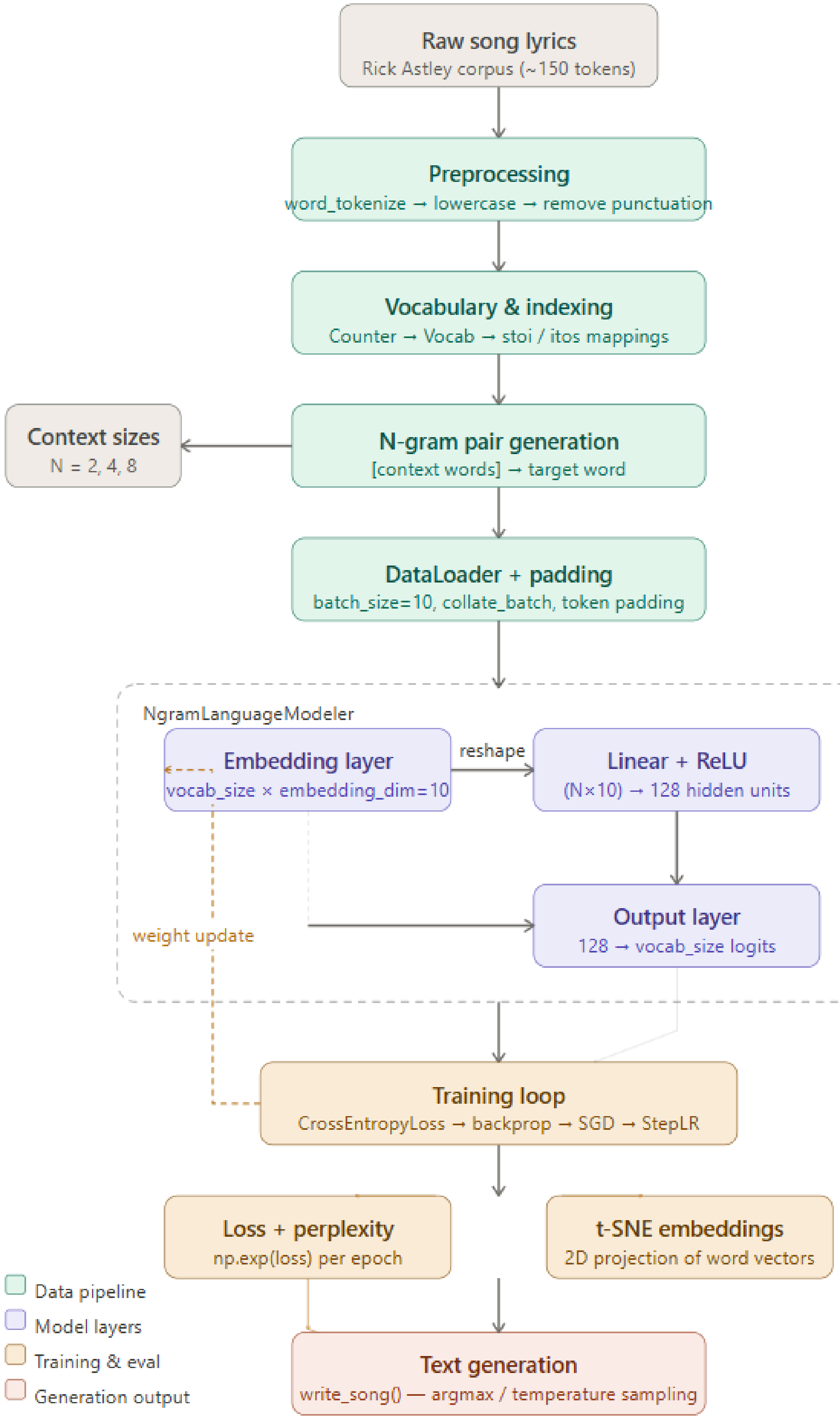
Key Limitations:

1. Language model trained on ~150 tokens and the smaller corpus causes word repetition even with 8 gram model - need larger corpus
2. Increasing N gram won't produce significant gains due to repetitive loops
3. Perplexity nears 1 for 8 gram model but due to small corpus it is due to memorization of tokens

What's next:

1. Rebuild this with a single attention head on the same corpus to see the delta directly.
2. Doing this alongside W&B MLOps as part of a deliberate push to go deeper on the ML foundations — understanding failure modes at the architecture level, not just the program level.

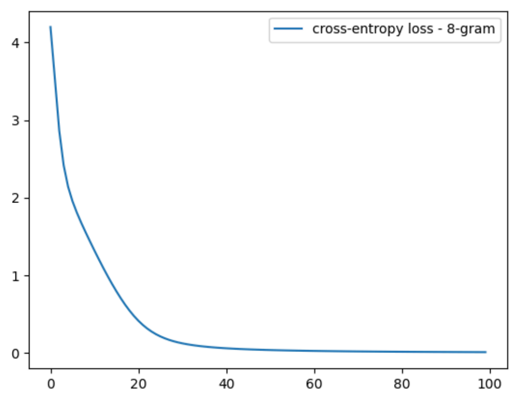
   Report here: https://wandb.ai/nishanth-anantha87-asml/ngram-song-generator/reports/FFN-for-Language-Model-N-gram-model-comparison-Song-generator---VmlldzoxNjUwMzA4NA

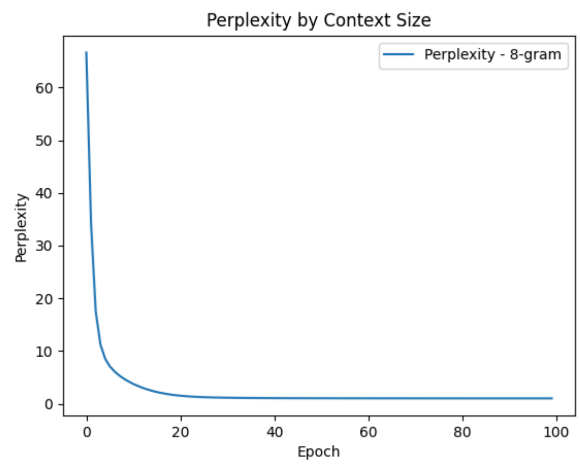


Raw song lyrics
Rick Astley corpus (~150 tokens)
Preprocessing
word_tokenize → lowercase → remove punctuation
Vocabulary & indexing
Counter → Vocab → stoi / itos mappings
Context sizes
N = 2, 4, 8
N-gram pair generation
[context words] → target word
DataLoader + padding
batch_size=10, collate_batch, token padding
NgramLanguageModeler
Embedding layer
vocab_size × embedding_dim=10
reshape
Linear + ReLU
(N×10) → 128 hidden units
weight update
Output layer
128 → vocab_size logits
Training loop
CrossEntropyLoss → backprop → SGD → StepLR
Loss + perplexity
np.exp(loss) per epoch
t-SNE embeddings
2D projection of word vectors
Text generation
write_song() — argmax / temperature sampling
Data pipeline
Model layers
Training & eval
Generation output

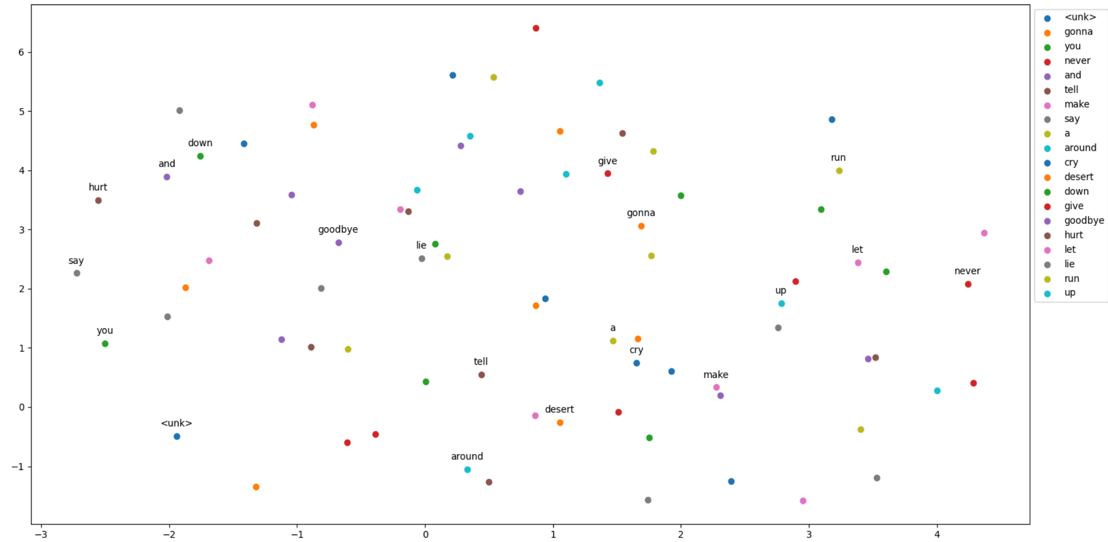
Key Findings

1. Larger context window reduces loss — but on a small corpus this is memorization The 8-gram model achieves near-zero loss and perplexity ~1.0. This looks like success but is actually the model memorizing the 150-token Rick Astley corpus rather than learning generalizable language patterns. With a context window of 8 words and a vocabulary of ~50 unique words, the model has seen nearly every possible sequence during training. **Insight:** On a real dataset (10M+ tokens), the 2-gram vs 8-gram difference would reflect genuine language understanding, not memorization. Corpus size matters as much as architecture.





2. The coherence wall — why this model fails after ~5 words Generating a 20-word sequence from each model and we observe: all three models produce repetitive loops ("never gonna never gonna never gonna...") within 5-8 words. This is the **fixed context window** problem in action. The model can only reference the previous N tokens. Once it generates a repeated sequence, it has no mechanism to "remember" that it already said "never gonna give you up" three lines ago. There's no long-range dependency. **This is exactly the problem attention mechanisms solve.** The transformer architecture allows any token to attend to any previous token regardless of distance. A transformer trained on this corpus would learn that "never gonna" almost always precedes one of five specific phrases — not because of the immediate context but because of global pattern recognition across the full sequence.

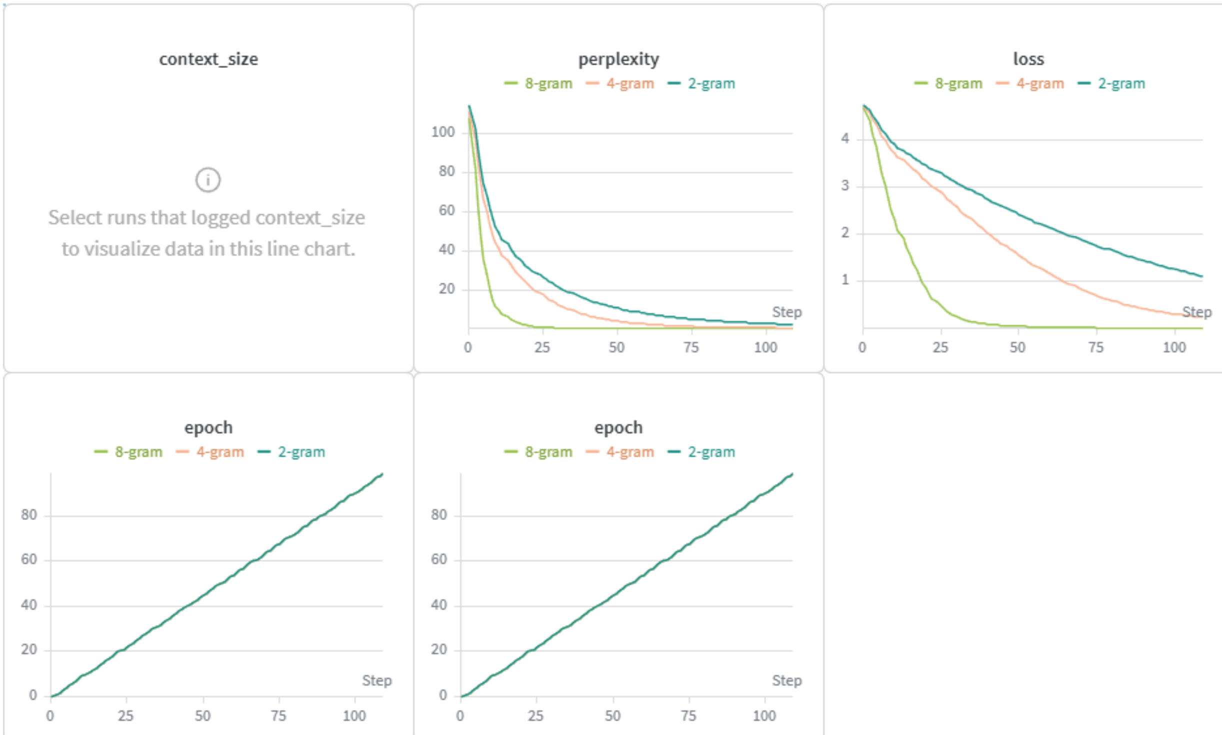


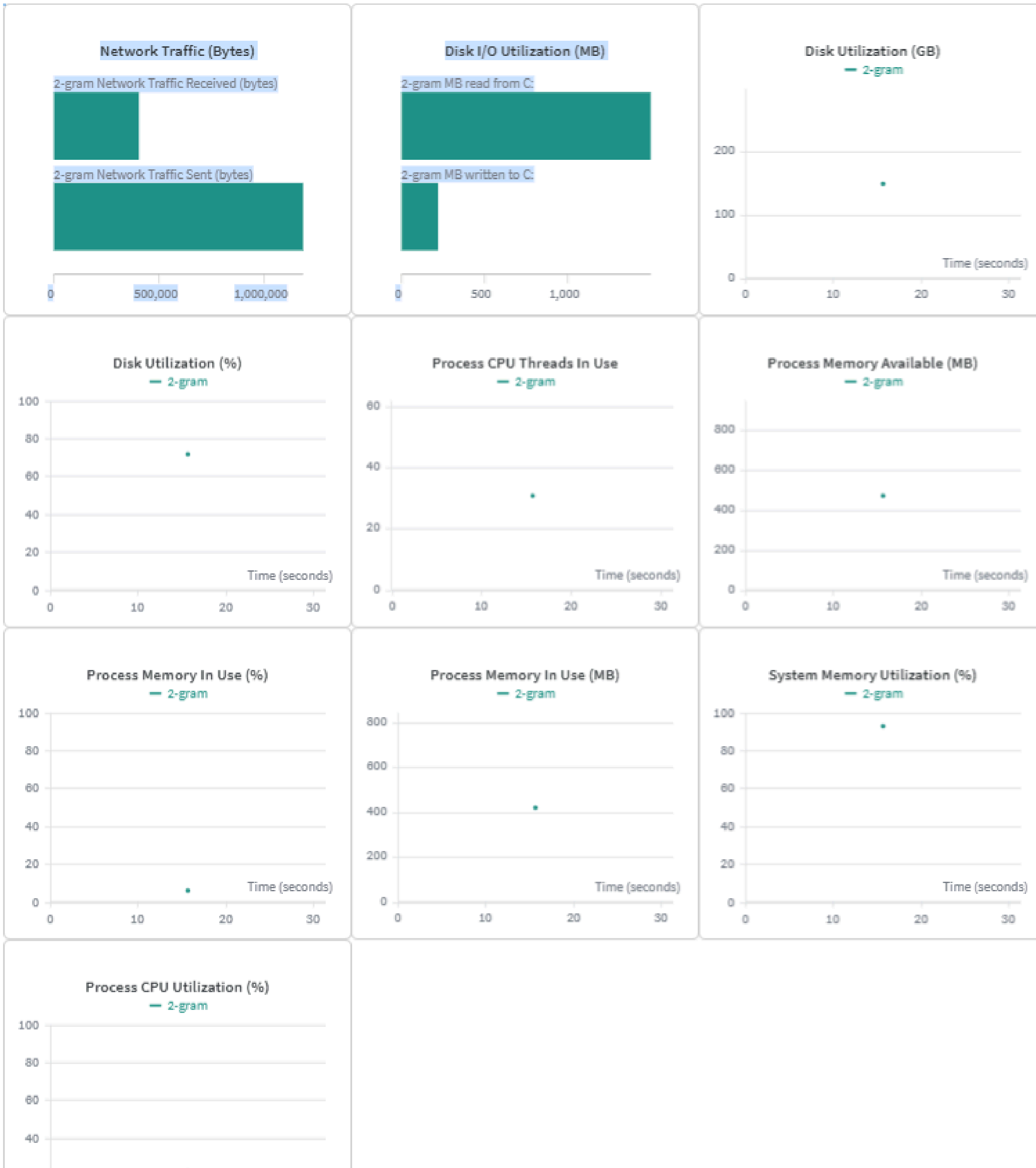
3. Embedding visualization shows learned semantic structure The t-SNE plots show the model learning word relationships purely from co-occurrence patterns. Action words ("give," "run," "tell," "make") cluster differently from emotional words ("hurt," "cry"). This emerges without any explicit semantic supervision — purely from predicting the next word. This is the same principle underlying word2vec and the embedding layers in every modern LLM, scaled to billions of tokens.

4. What would make this better In order of impact:
1. Larger corpus — train on a full album or genre dataset from Kaggle
2. Validation split — current perplexity is on training data; held-out perplexity would reveal actual generalization
3. Beam search instead of argmax — `torch.argmax` always picks the single most likely word, producing repetitive output; beam search or temperature sampling would generate more varied text
4. The real fix: attention — a transformer with even 1 attention head would outperform the 8-gram FNN on any corpus larger than this song

Weights and Biases Experiment with plot comparisons among N gram models (with dashboard)

Report here:
https://wandb.ai/nishanth-anantha87-asml/ngram-song-generator/reports/FFN-for-Language-Model-N-gram-model-comparison-Song-generator---VmlldzoxNjUwMzA4NA


context_size
Select runs that logged context_size to visualize data in this line chart.
perplexity
8-gram 4-gram 2-gram
Step
loss
8-gram 4-gram 2-gram
Step
epoch
8-gram 4-gram 2-gram
Step
epoch
8-gram 4-gram 2-gram
Step


Network Traffic (Bytes)
2-gram Network Traffic Received (bytes)
2-gram Network Traffic Sent (bytes)
0
500,000
1,000,000
Disk I/O Utilization (MB)
2-gram MB read from C:
2-gram MB written to C:
0
500
1,000
Disk Utilization (GB)
2-gram
Time (seconds)
Disk Utilization (%)
2-gram
Time (seconds)
Process CPU Threads In Use
2-gram
Time (seconds)
Process Memory Available (MB)
2-gram
Time (seconds)
Process Memory In Use (%)
2-gram
Time (seconds)
Process Memory In Use (MB)
2-gram
Time (seconds)
System Memory Utilization (%)
2-gram
Time (seconds)
Process CPU Utilization (%)
2-gram